

Lectures – 21, 22, 23

SHORTEST PATH ALGORITHMS

Consider a weighted, directed graph $G = (V, E)$ with weight function w ; the weight function $w: E \rightarrow \mathbb{R}$ is similar to what we have seen in MST algorithms.

Let $p = \langle v_0, v_1 \dots v_k \rangle$ be a path in G . Then the weight $w(p)$ of this path is defined as:

$$w(p) = \sum_{i=1, k} w(v_{i-1}, v_i)$$

The **shortest path weight** $\delta(u, v)$, for the pair of vertices u and v , is defined as:

- (1) the minimum of the weights of all paths from u to v , if there is at least one such path, or
- (2) ∞ , if no path exists in G from u to v .

A **shortest path** in G from vertex u to vertex v is any path p with weight $w(p) = \delta(u, v)$.

Variants of the shortest path problem:

- Single-source shortest path problem,
- Single-destination shortest path problem,
- Single-pair shortest path problem, and
- All-pairs shortest path problem.

Optimal sub-structure of the shortest path problem:

With G and w as above, let $p = \langle v_1, v_2 \dots v_k \rangle$ be a shortest path in G from v_1 to v_k . Further, for any i, j such that $1 \leq i \leq j \leq k$, let p_{ij} be the sub-path of p from v_i to v_j . **Then p_{ij} is a shortest path from v_i to v_j .**

In simple words, any sub-path of a shortest path is also a shortest path. For our purposes, we also assume that G contains **no negative weight edges**.

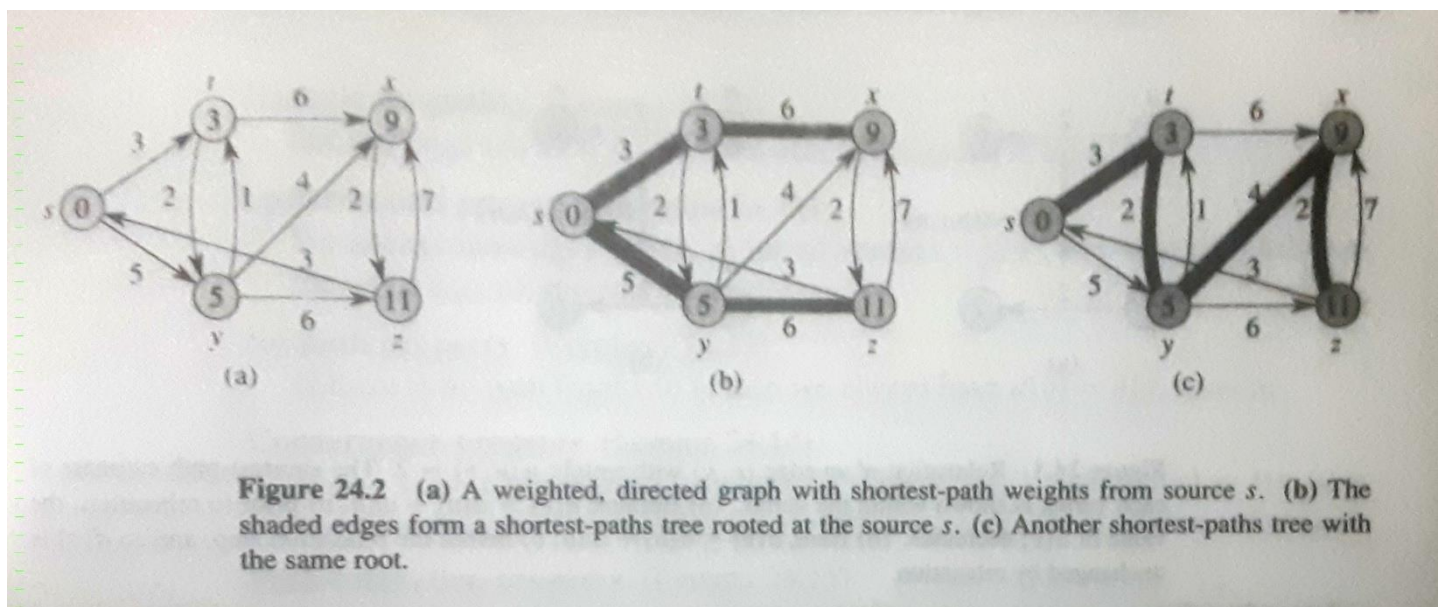
[Note that a **greedy algorithm** gives the correct solution to a problem only when the problem has **optimal sub-structure** of the type seen above.]

Shortest paths will be represented using **predecessor subgraphs**, as we have done with MST algorithms. That is, for any vertex v , the vertex $\pi[v]$ or $\text{pred}[v]$ is the predecessor of v along a shortest path.

With G and w as above, let s be a specified source vertex. A **shortest path tree rooted at s** is a directed subgraph $G' = (V', E')$ of G such that:

1. V' is the set of all vertices reachable from s in G ,
2. G' is a rooted tree with root s , and
3. For all the vertices v in V' , the unique simple path from s to v in G' is the shortest path from s to v in G .

Example:



Relaxation

Here $d[v]$ is an attribute of every vertex v in V , which is an upper bound on the weight of the shortest path from s to v . We call it **shortest path estimate**. It is initialized to ∞ for all the vertices except s , for which it is set to 0.

RELAX (u, v, w)

```

if  $d[v] > d[u] + w(u, v)$ 
     $d[v] = d[u] + w(u, v)$ ; //  $d[v]$  and  $\text{pred}[v]$ 
     $\text{pred}[v] = u$ ;          // are updated

```

[As for MST algorithms, the predecessor $\pi[v]$ of every vertex v in V is initialized to NULL. Recall the almost identical step in Prim's algorithm.]

All the algorithms which we shall study in this chapter make use of **relaxation**.

Properties of shortest paths and relaxation

Triangle inequality: For any edge (u,v) , $\delta(s,v) \leq \delta(s,u) + w(u,v)$.

Upper bound property: $d[v] \geq \delta(s,v)$. If v is reachable from s , $d[v]$ finally reaches the value $\delta(s,v)$.

No path property: If there is no path from s to v , then $d[v] = \delta(s,v) = \infty$.

Convergence property: Let $s \rightsquigarrow u \rightarrow v$ be a shortest path in G for some vertices u, v ; and let $d[u] = \delta(s,u)$ at some stage prior to relaxing the edge (u,v) . Then $d[v] = \delta(s,v)$ at all times after relaxing the edge (u,v) .

Path relaxation property: Let $p = \langle s, v_1, v_2 \dots v_k \rangle$ be a shortest path from s to v_k , and let the edges along that path be relaxed in the order they occur on the path. Then $d[v_k] = \delta(s,v_k)$, regardless of the order of relaxation of other edges.

Predecessor subgraph property: Once $d[v] = \delta(s,v)$ for all the vertices in V , the predecessor subgraph is the shortest path tree rooted at s .

The following is a straightforward function for initialization:

```
INITIALIZE-SINGLE-SOURCE( $G, s$ )
  for each vertex  $v$  in  $V[G]$ 
     $d[v] = \infty$ ;
     $pred[v] = NIL$ ;
   $d[s] = 0$ ;
```

The standard single-source shortest path algorithm:

```
BELLMAN-FORD( $G, w, s$ )
  INITIALIZE-SINGLE-SOURCE( $G, s$ ) ;
  for  $i = 1$  to  $|V[G]| - 1$ 
    for each edge  $(u,v)$  in  $E[G]$ 
      RELAX( $u, v, w$ ) ;
```

Bellman-Ford algorithm can handle negative weight edges, but of course no algorithm works if there are negative weight cycles in G which are reachable from the source s .

Running time: $O(VE)$, that is, $O(|V| \times |E|)$

[Follows from the 'E loop' inside 'V loop' structure.]

Self-study:

Solved example of Fig. 24.4. **Note the effect of negative weight edges.**

SINGLE-SOURCE SHORTEST PATHS IN **DAGS**

DAG-SHORTEST-PATHS (G, w, s)

```
Topologically sort the vertices of G;  
INITIALIZE-SINGLE-SOURCE ( $G, s$ ) ;  
for each vertex  $u$  in  $G$ , in topological order  
  for each vertex  $v$  in  $ADJ[u]$   
    RELAX ( $u, v, w$ ) ;
```

Running time: $\Theta(V+E)$. Note that every edge is processed only once.

Because the graph is acyclic, negative edge weights do not matter.

Example:

See below [from CLRS].

Questions:

When would 'the longest path' in a DAG have practical meaning?

How can we generate a randomized DAG?

In the following solved example, topologically sorted vertices appear in order from left to right.

Note that there is no path in G from source s to the leftmost vertex r ; so the final shortest path tree does not include vertex r .

